K-뷰티 일본 진출 지원 서비스 기술 스택 분석

Oct 21, 2026 · @Min

읽기 전에: 결론과 전제

결론부터 말하면 **Next.js(TypeScript) + Supabase + Claude API**로 시작하고, 리뷰·랭킹 수집만 Python 스크립트로 따로 돌리는 구조를 추천합니다. 아래 단계는 이 결론이 요구사항에서 어떻게 나오는지를 보여줍니다.

전제는 기획안에서만 가져왔습니다. 50~100명 규모 회사의 해외사업팀이 쓰고, 개발 인력이 많지 않으며, 빨리 써 보고 고쳐 나가야 합니다. 그래서 모든 단계에서 "사람이 적어도 굴러가고, 운영 부담이 적은 쪽"을 우선했습니다.

해석 하나: 기획안의 "시장트렌드분석 (GEO)"를 **Generative Engine Optimization**, 즉 ChatGPT·Gemini 같은 AI 검색이 일본 화장품을 물었을 때 어떤 브랜드를 추천하는지 보는 분석으로 읽었습니다. 지역(geography) 분석을 뜻했다면 알려주세요. 해당 부분만 바꾸면 됩니다.

1~2단계. 무엇을 위한 프로그램이고, 어디서 쓰는가

한 문장 정의: 일본 진출을 준비하는 K-뷰티 중소기업 해외마케팅팀이 시장 조사, 리뷰 분석, 약기법 광고 검토, 프로모션 일정을 한 곳에서 AI로 처리하는 팀 업무용 웹서비스입니다.

사용 환경은 **웹**입니다. 다만 사용자가 두 부류라서 화면도 두 갈래로 나뉩니다.

사용자	어디서	왜 그렇게 판단했나
해외마케팅팀 (주 사용자)	회사 PC 브라우저	표, 차트, 긴 광고 문구를 다루는 사무 업무입니다. 팀원끼리 결과를 공유해야 하므로 설치가 필요 없는 웹이 맞습니다.
일본 소비자 (피부타입 설문, 기능 5)	스마트폰 브라우저	SNS나 쇼핑몰 링크로 들어와 1~2분 안에 끝내는 설문입니다. 앱 설치를 요구하면 아무도 하지 않으므로 모바일 웹이 맞습니다.

화면 뒤에는 **서버**도 필요합니다. 리뷰 수집, AI 호출, 팀 데이터 저장은 브라우저가 혼자 할 수 없는 일이기 때문입니다. 모바일 앱, 데스크톱 앱, 임베디드는 필요하지 않습니다.

비유하면 이 서비스는 회사 안의 **다리실**(사내 대시보드)과 거리에 둔 **시음대**(설문 페이지)를 한 건물에서 운영하는 것과 같습니다. 출입문은 다르지만 주방과 창고는 같이 씁니다.

3~4단계. 핵심 기능과 기능별로 필요한 기술 종류

기획안의 3개 탭과 "특별히 신경 쓰고 싶은 기능" 8개를 합치면 아래 11개 기능으로 묶입니다. 여기서는 아직 제품 이름을 쓰지 않고, "어떤 종류의 기술이 필요한가"만 적습니다.

#	기능	실제로 하는 일	필요한 기술 종류
A	약기법 광고 문구 검토 (우선순위 1, TAB 3)	문구 입력 → 위험 표현 표시 → 사유와 규제 근거 → 대체 표현 제안 → 검토 이력 저장	AI(글 이해·생성), 규제 지식 문서 참조, 구조화된 출력(JSON), DB
B	시장 트렌드 + GEO (우선순위 2, TAB 1)	랭킹·검색량·성분 언급 추이, AI 검색이 추천하는 브랜드 기록	외부 API·크롤링, 정기 실행(스케줄러), 차트
C	경쟁제품 비교 (우선순위 2, TAB 1)	가격·용량·성분·특징을 표로 나란히 비교	상품 페이지·이미지에서 정보 추출(멀티모달 AI), 표 형태 저장
D	일본 현지 제품 리뷰 분석 (우선순위 3)	@cosme·라쿠텐·Qoo10 리뷰 수집 → 감성·키워드·불만 유형 분석	크롤링, AI 분류, 일본어 텍스트 처리, 대량 처리(배치)
E	국내 리뷰 분석 (TAB 2)	올리브영 등 리뷰 수집 또는 CSV 업로드 → 감성·키워드·페인포인트·재구매 요인 → 차트 → 원문 연결	파일 업로드, 크롤링, AI 분류, 차트, 분석 결과와 원문을 잇는 DB 구조
F	한국 리뷰 → 일본어 상세페이지 (우선순위 4)	좋은 리뷰를 골라 일본어로 번역하고 상세페이지용 블록으로 내보냄	AI 번역, 약기법 검토(A) 재사용, HTML/이미지 내보내기
G	피부타입 설문 (우선순위 5)	일본 소비자용 공개 설문 → 결과 저장·집계	모바일 화면, 로그인 없는 공개 페이지, 개인정보 동의, DB
H	플랫폼별 강점 (우선순위 6)	라쿠텐·Qoo10·아마존·@cosme 등 특징 정리	대부분 콘텐츠 작업. 기술은 화면과 DB면 충분
I	메가와리·프로모션 캘린더 (우선순위 7)	행사 일정 등록·공유, 준비 일정 역산	캘린더 화면, DB
J	팀 워크스페이스 (우선순위 8)	로그인, 팀별 데이터 분리, 모든 분석 결과 보관	인증(로그인), 권한 관리, 관계형 DB
K	공통 화면	탭 구조의 대시보드	웹 프론트엔드, UI 컴포넌트

이번 버전에서 **필요 없는 것**도 정리해 둡니다. 결제(팀 내부 도구이므로), 실시간 통신(동시 편집이 핵심이 아님), 별도 관리자 시스템(DB 콘솔로 대신)은 빼도 됩니다.

눈여겨볼 점 두 가지:

- **F는 A와 연결됩니다.** 판매자가 상세페이지에 올린 리뷰는 일본에서 광고로 취급될 수 있습니다. 리뷰에 "미백 효과", "여드름이 나았다" 같은 표현이 있으면 가지고 올리면 위험합니다. 그래서 번역된 리뷰는 약기법 검토를 거치게 설계하면, 탭 간 연계라는 차별점도 자연스럽게 생깁니다. (정확한 범위는

전문가 확인이 필요합니다.)

- **D와 E는 같은 기계입니다.** 언어(일본어/한국어)와 데이터 출처만 다르고 분석 흐름은 같으므로, 하나의 "리뷰 분석 공장"을 만들고 재료만 바꿔 넣으면 됩니다.

5~6단계. 언어 선택, 그다음 라이브러리

언어는 **두 개**입니다. 화면과 서버는 TypeScript, 데이터 수집은 Python입니다. 하나로 통일하면 더 단순하지만, 각각의 일에서 얻는 이점이 커서 둘로 나눌습니다.

언어 결정 흐름

1. **팀원이 브라우저에서 표·차트·캘린더를 다루는 화면이 필요하다** → 브라우저가 알아듣는 언어는 사실상 JavaScript 하나뿐이다 → 그 위에 "자료형 검사"를 얹은 **TypeScript**를 선택한다.
   - 자료형 검사란: "가격 칸에는 숫자만" 같은 규칙을 미리 적어 두고, 어기면 실행 전에 빨간 줄로 알려주는 기능입니다. 요리 전에 재료 라벨을 확인하는 것과 같습니다.
2. **AI를 부르고 팀 데이터를 저장하는 서버가 필요하다** → 이 서비스의 서버 일은 "요청 받기 → AI 호출 → DB 저장"이라 무거운 계산이 없다. 주요 AI 회사들은 TypeScript용 공식 SDK를 제공한다 → 화면과 같은 **TypeScript**로 서버도 쓴다. 한 프로젝트에 한 언어면 팀원 누구나 양쪽을 고칠 수 있다.
3. **여러 플랫폼에 흩어진 리뷰와 랭킹을 모아야 한다(기획안의 첫 페인포인트)** → 크롤링과 데이터 정리는 Python 생태계가 가장 두텁고 예제도 한국어 자료도 가장 많다. 일본어 형태소 분석기(SudachiPy)도 Python이 표준이다 → 수집·전처리 스크립트는 **Python**으로 따로 쓴다.
   - 형태소 분석이란: "保湿力が高い"를 "保湿 / 力 / が / 高い"처럼 단어 단위로 자르는 일입니다. 일본어는 띄어쓰기가 없어서 키워드 빈도를 세려면 이 칼질이 먼저입니다.

비유: 식당 홀과 주방은 같은 말(TypeScript)을 쓰는 직원들이 운영하고, 새벽 시장에서 재료를 떼오는 구매 담당(Python)은 자기에게 가장 편한 도구를 씁니다. 둘은 직접 대화하지 않고, 창고(DB)에 재료를 넣고 꺼내는 것으로만 연결됩니다.

라이브러리와 프레임워크

프레임워크는 집의 뼈대(설계 규칙 포함), 라이브러리는 사다 쓰는 가구라고 생각하면 됩니다.

기술	언어	왜 필요한가 (기능 번호)
Next.js	TS	화면(K)과 서버 API를 한 프로젝트에서 만듭니다. 사내 대시보드와 공개 설문(G)을 같은 앱의 다른 주소로 나눌 수 있습니다.
Tailwind CSS + shadcn/ui	TS	버튼·표·탭·모달 같은 화면 부품을 디자이너 없이도 깔끔하게 빠르게 만듭니다.

기술	언어	왜 필요한가 (기능 번호)
Recharts	TS	감성 비율, 키워드 빈도, 트렌드 추이 차트(B, D, E)를 그립니다.
FullCalendar	TS	메가와리·슈퍼세일 일정을 월/주 단위 달력으로 보여줍니다(I). 달력을 직접 만드는 건 생각보다 어렵습니다.
Papa Parse	TS	업로드한 리뷰 CSV를 표 데이터로 읽습니다(E).
Anthropic SDK (@anthropic-ai/sdk)	TS	Claude 호출 통로입니다. 구조화된 JSON 출력, 이미지 입력을 여기서 씁니다(A~F).
Zod	TS	AI가 돌려준 JSON이 약속한 모양인지 검사합니다. AI가 가끔 하는 실수를 화면에 내보내기 전에 걸러냅니다.
Supabase JS	TS	로그인, DB 읽기·쓰기, 파일 저장을 한 도구로 처리합니다(J).
Playwright (Python)	Py	자바스크립트로 그려지는 쇼핑몰 리뷰 페이지를 실제 브라우저처럼 열어 수집합니다(D, E).
requests + BeautifulSoup	Py	단순한 페이지와 라쿠텐 API처럼 가벼운 수집에 씁니다. Playwright보다 빠르고 가볍습니다(B, C).
pandas	Py	수집한 데이터의 중복 제거, 날짜 정리, 표 변환을 합니다.
SudachiPy	Py	일본어 리뷰를 단어로 자르고 빈도를 셉니다(D). 키워드 빈도를 전부 AI에 맡기면 비용이 커지고 숫자가 매번 바뀝니다.

트렌드·경쟁사 데이터는 가능하면 **공식 API를 먼저** 씁니다. 라쿠텐은 상품 검색·랭킹 API를 공개하고 있어 크롤링보다 안정적입니다. @cosme와 Qoo10은 크롤링이 필요할 텐데, 각 사이트 이용약관과 robots.txt를 확인하고 요청 간격을 둔 소량 수집으로 한정하는 것이 안전합니다. (사이트별 약관은 이번에 직접 확인하지 않았습니다.)

7~8단계. 데이터베이스, 그리고 서버를 직접 운영할지

관계형 DB(PostgreSQL) 하나로 시작합니다. NoSQL은 필요 없고, Vector DB는 규제 문서가 많아지면 같은 DB에 확장 기능으로 붙입니다.

DB 종류 판단

종류	비유	필요한가	이유
관계형 DB	칸과 라벨이 정해진 물류 창고. "팀 → 제품 → 리뷰 → 분석 결과"처럼 선반끼리 연결됨	**예, 처음부터**	이 서비스의 데이터는 거의 다 표입니다. 경쟁사 비교(가격·용량·성분), 캘린더 일정, 검토 이력이 그렇습니다. "이 분석 결과의 근거 리뷰 원문 보기"(E)는 표끼리 연결하는 관계형 DB의 특기입니다. 팀별 데이터 분리(J)도 여기서 가장 안전하게 됩니다.
NoSQL	아무 모양 상자나 넣는 창고	아니오	크롤링한 원본처럼 모양이 제각각인 데이터는 PostgreSQL의 JSONB 칸에 넣으면 됩니다. 창고 두 개를 관리할 이유가 없습니다.
Vector DB	단어가 아니라 뜻으로 책을 찾아주는 사서	**나중에**	약기법 검토(A)의 핵심 자료인 화장품 효능 범위(56개 항목)와 적정광고 가이드라인의 NG 표현은 프롬프트에 통째로 넣을 수 있는 분량입니다. 적발 사례, 과거 검토 이력, 리뷰 수만 건에서 비슷한 말 찾기가 필요해지면 그때 PostgreSQL의 pgvector를 켭니다.

용어 하나: **RAG**(검색 증강 생성)는 AI가 답하기 전에 관련 문서를 먼저 찾아 손에 쥐어주는 방식입니다. 오픈북 시험에서 필요한 페이지를 펼쳐주는 것과 같습니다. 책이 얇으면 통째로 주면 되고(MVP), 두꺼워지면 사서(Vector DB)가 필요해집니다.

추가로 파일 저장소(리뷰 CSV, 경쟁사 상품 이미지)가 필요합니다. 이건 DB가 아니라 창고 옆 컨테이너 야적장입니다.

직접 운영 vs 관리형 서비스

비유하면 직접 운영은 **건물을 사서 전기·배관·보안까지 직접 관리**하는 것이고, 관리형은 **관리사무소가 있는 공유오피스에 입주**하는 것입니다.

선택지	이 프로젝트에 맞는 점	안 맞는 점	판정
Supabase	PostgreSQL + 로그인 + 파일 저장 + pgvector를 한 곳에서 제공. 팀별 데이터 분리 규칙(RLS)을 DB에 직접 건다	무료 플랜은 일주일 미사용 시 일시 정지됨	**DB·로그인용으로 채택**

선택지	이 프로젝트에 맞는 점	안 맞는 점	판정
Vercel	Next.js 제작사라 배포가 GitHub 연결 한 번으로 끝남. 푸시할 때마다 미리보기 주소 생성	서버 함수 실행 시간에 제한이 있어 긴 크롤링은 여기서 못 돌림	**웹 배포용으로 채택**
GitHub Actions	정해진 시간에 Python 수집 스크립트를 돌리는 무료 스케줄러. 코드 저장소와 같은 곳이라 설정이 간단함	실행 시간이 정확하지 않고, 서비스용 인프라로는 장기적으로 어색함	**MVP 수집 스케줄러로 채택**
Firebase	로그인·호스팅이 쉽고 Google 생태계	DB가 NoSQL(Firestore)이라 경쟁사 비교표, 리뷰-분석 연결 같은 표 중심 데이터에 불리	보류
AWS (직접 구성)	무엇이든 가능, 크게 키우기 좋음	설정할 것이 많고 비용 구조가 복잡해 초급 팀에게 시간 도둑	서비스가 커진 뒤 검토
직접 서버 운영 (VPS)	월 비용이 가장 저렴	보안 패치, 백업, 장애 대응을 팀이 맡아야 함	제외

결론: **Vercel**(화면+서버), **Supabase**(DB·로그인·파일), **GitHub Actions**(수집 스케줄) 조합입니다. 세 가지 모두 무료 플랜으로 초기 운영까지 갈 수 있고, 서버 관리 업무가 거의 없습니다. (무료 플랜 한도는 수시로 바뀌므로 시작 전에 확인하세요.)

9단계. 전체 시스템 구조

요청은 위에서 아래로 흐르고, 수집은 오른쪽 줄에서 따로 돌아 같은 DB에 쌓입니다. 두 줄이 만나는 곳은 DB뿐입니다.



```
        |  팀 · 제품 · 리뷰 · 분석결과 · 검토이력     |
        |  · 캘린더 · 설문응답 (+ 파일 저장소)        |
                                                    |
                Python 수집 스크립트 (GitHub Actions, 매일)
                          ↑
      External: 라쿠텐 API · @cosme · Qoo10 · 올리브영
                · AI 검색 응답(GEO)
```

공장으로 비유하면 오른쪽 줄은 **원자재 입고 라인**(매일 밤 트럭이 리뷰와 랭킹을 창고에 부려 둠), 왼쪽 줄은 **주문 생산 라인**(팀원이 버튼을 누르면 창고 재료를 꺼내 AI가 가공)입니다. 입고가 멈춰도 이미 쌓인 재료로 생산은 계속됩니다.

10~11단계. 최종 추천 기술 스택

처음부터 쓰는 기술은 열 개 안팎입니다. 표의 각 행이 "역할 · 선택 이유 · 장점 · 단점 · 대체 기술"을 한 번에 보여줍니다.

Frontend

기술	역할	선택 이유	장점	단점	대체 기술
Next.js (TypeScript)	대시보드와 설문 페이지 화면	화면과 서버를 한 프로젝트·한 언어로	자료가 많고 Vercel 배포가 쉬움	기능이 많아 처음엔 개념이 헷갈림	Vite + React, Streamlit(Python만으로 빠르게, 대신 화면 자유도 낮음)
Tailwind + shadcn/ui	화면 부품과 디자인	디자이너 없이도 정돈된 UI	복사해서 쓰는 부품이라 수정이 자유로움	클래스 이름이 길어져 코드가 지저분해 보임	MUI, Chakra UI
Recharts	감성·키워드·트렌드 차트	React 화면에 바로 끼워 쓰는 차트	사용법이 단순	아주 큰 데이터에서는 느려짐	Chart.js, ECharts
FullCalendar	프로모션 캘린더	달력 UI를 직접 만들지 않기 위해	월·주·리스트 보기, 드래그 이동	고급 기능 일부는 유료	react-big-calendar

Backend

기술	역할	선택 이유	장점	단점	대체 기술
Next.js 서버 API (Route Handler)	로그인 확인, Gemini 호출, DB 저장	서버 일이 가볍고 화면과 언어가 같음	별도 서버 불필요	한 요청당 실행 시간 제한이 있음(리뷰 수천 건은 나눠 처리)	FastAPI(Python), Express

기술	역할	선택 이유	장점	단점	대체 기술
Zod	AI 응답 모양 검사	Gemini JSON 출력을 신뢰하기 전 검문	검사 규칙을 Gemini 스키마로도 재사용	없음 수준	Valibot
Python 수집 스크립트 (Playwright, BeautifulSoup, pandas, SudachiPy)	리뷰·랭킹 수집, 일본어 키워드 빈도	크롤링과 일본어 처리는 Python이 가장 편함	예제 풍부, 일본어 처리 도구가 갖춰짐	사이트 구조가 바뀌면 수집이 깨짐, 약관 확인 필요	Node.js용 Playwright, Apify 같은 수집 대행 서비스

Database

기술	역할	선택 이유	장점	단점	대체 기술
Supabase PostgreSQL	팀·제품·리뷰·분석·이력·일정·설문 저장	데이터가 표 형태이고 서로 연결됨	로그인·파일·RLS·pgvector가 한 곳에	SQL을 조금은 배워야 함	Neon, Firebase(NoSQL), PlanetScale
Supabase Storage	리뷰 CSV, 상품 이미지 보관	DB와 같은 권한 규칙 적용	추가 계정 불필요	대용량이면 비용 상승	AWS S3, Cloudflare R2

AI

기술	역할	선택 이유	장점	단점	대체 기술
Claude API, Sonnet 계열 모델	약기법 위험 판단, 대체 표현, 페인포인트 요약	긴 규제 문서를 읽고 일본어 뉘앙스를 판단해야 하므로 긴 문서 이해와 글 품질이 중요	넓은 입력창, 근거 설명이 꼼꼼함, 구조화된 JSON 출력	대량 처리엔 비용이 큼	Gemini Pro, GPT
Claude API, Haiku 계열 모델	리뷰 감성 분류, 키워드, 번역, 상품 이미지에서 성분·용량 추출	수천 건 반복 작업은 빠르고 싼 모델로	속도·비용, 이미지 입력	미묘한 판단은 Sonnet보다 약함	Gemini Flash, GPT mini 계열
프롬프트 설계 (예시 제공, 판단 단계 지시, JSON 스키마)	결과의 일관성과 근거 제시	같은 문구에 매번 다른 판정이 나오면 실무에서 못 씀	코드 없이 품질 향상	프롬프트 버전 관리가 필요	-

AI에 필요한 조건은 네 가지입니다. 긴 규제 문서 이해, 일본어 글 품질, 이미지 입력, 정해진 모양의 JSON 출력입니다. Claude는 이를 모두 만족하는 선택지 중 하나이고, Gemini나 GPT로 바꿔도 전체 구조는 그대로입니다. 모델 이름은 버전이 자주 바뀌므로 계열로만 적었고, 시작 시점의 최신 버전을 고르면 됩니다.

프롬프트의 "예시 제공"(Few-shot)은 "이런 문구는 이렇게 판정한다"는 샘플을 몇 개 함께 주는 것이고, "판단 단계 지시"는 결론 전에 체크항목을 하나씩 확인하게 하는 것입니다. 새 직원에게 완성 예시를 보여주고 체크리스트대로 검수하게 하는 것과 같습니다.

Infrastructure / Deploy

기술	역할	선택 이유	장점	단점	대체 기술
Vercel	웹 배포	Next.js와 짝이 가장 잘 맞음	push하면 자동 배포, 미리보기 주소	긴 작업 실행 불가, 트래픽 커지면 비용	Netlify, Cloudflare Pages, AWS Amplify
GitHub Actions	매일 수집 스크립트 실행	코드 제출처와 같은 곳, 추가 설정 최소	무료, 실행 기록이 남아 수집 실패를 찾기 쉬움	정시 실행 보장 없음	Supabase Cron, Cloud Run Jobs
GitHub	코드·프롬프트 관리	팀 협업의 기본 도구	변경 이력과 리뷰가 남음	-	GitLab

12~13단계. MVP로 시작하고 필요할 때 늘리기

MVP는 "핵심 가치를 보여줄 수 있는 가장 작은 완성품"입니다. 식당으로 치면 메뉴 50개로 개업하지 않고, 대표 메뉴 3개를 제대로 내는 팝업 매장으로 시작하는 것입니다.

처음부터 필요한 것 vs 커진 뒤 추가할 것

구분	처음부터 (MVP)	커진 뒤 추가	추가하는 신호
화면	Next.js, Tailwind/shadcn, Recharts, FullCalendar	다국어 UI(i18n)	일본 현지 파트너도 쓸 때
서버	Next.js 서버 API, Zod	백그라운드 작업 큐(Inngest, Trigger.dev)	리뷰 수천 건 분석이 시간 제한에 걸릴 때
데이터 수집	Python 스크립트 + GitHub Actions, 라쿠텐 API, CSV 업로드	전용 수집 서버(FastAPI + Cloud Run), 프록시	수집 대상이 수십 개로 늘 때
DB	Supabase PostgreSQL + Storage + RLS	pgvector(Vector 검색), 읽기 전용 복제본	약기법 사례·검토 이력이 프롬프트에 다 안 들어갈 때
AI	Claude Sonnet/Haiku, 예시·판단 단계·JSON 스키마 프롬프트, 규제 문서 통째 프롬프트	RAG(임베딩 모델 + pgvector), 프롬프트 평가 자동화	오판 사례가 쌓일 때

구분	처음부터 (MVP)	커진 뒤 추가	추가하는 신호
운영	Vercel, GitHub	오류 추적(Sentry), 사용 분석, 결제(Stripe)	외부 고객사가 생길 때

MVP 범위와 만드는 순서

우선순위와 "외부 데이터 없이도 바로 돌아가는가"를 함께 보고 순서를 정했습니다.

1. **뼈대**: 로그인, 팀 워크스페이스(J), 탭 3개짜리 빈 대시보드. 이후 모든 기능이 여기에 저장되므로 가장 먼저입니다.
2. **약기법 검토(A)**: 문구 입력만 있으면 돌아가고 우선순위 1이며 효과를 가장 빨리 확인할 수 있습니다. 규제 기준 문서를 프롬프트에 넣고 JSON으로 위험 표현·근거·대체안을 받습니다.
3. **리뷰 분석 공장(D, E)**: CSV 업로드로 먼저 분석 흐름을 완성하고, 그다음 Python 수집을 붙입니다. 재료가 없어도 기계는 먼저 만들 수 있습니다.
4. **리뷰 번역 → 상세페이지(F)**: 3의 결과에 번역과 2의 약기법 검토를 엮습니다. 탭 간 연계를 보여주는 기능입니다.
5. **경쟁제품 비교 + 트렌드(B, C)**: 라쿠텐 API로 랭킹·상품 정보를 받고, 상품 이미지는 Claude 이미지 입력으로 성분·용량을 뽑습니다. GEO는 정해진 질문을 AI 검색에 주기적으로 던져 브랜드 언급을 세는 방식으로 시작합니다.
6. **캘린더(I), 플랫폼 강점(H), 피부타입 설문(G)**: 기술 난도가 낮아 마지막에 빠르게 붙입니다. 설문은 일본 개인정보보호법에 맞춘 동의 문구가 필요합니다.

팀 역할을 나눌 때도 이 구조가 편합니다. 화면 담당, 서버·프롬프트 담당, Python 수집 담당이 DB 표 설계만 먼저 합의하면 동시에 작업할 수 있습니다.

약기법 기능의 결과는 법률 자문이 아니라 "검토가 필요한 표현 표시"임을 화면에 분명히 적고, 최종 판단은 담당자·전문가가 내리게 하는 것이 안전합니다. 기획안의 "담당자 검토 결과 저장" 기능이 바로 그 역할을 합니다.

14~15단계. 용어 비유 정리와 핵심 원칙

문서에 나온 어려운 말을 하나의 식당·물류 비유로 모았습니다.

용어	비유	이 서비스에서는
Frontend	손님이 앉는 홀과 메뉴판	대시보드, 설문 화면
Backend	주문을 받아 처리하는 주방	Next.js 서버 API
API	주방과 홀 사이의 주문서 양식	"이 문구 검토해줘" 요청
외부 API	외부 납품업체의 공식 주문 창구	라쿠텐 API, Claude API

용어	비유	이 서비스에서는
크롤링	시장에 직접 가서 가격표를 받아적는 조사원	@cosme·Qoo10 리뷰 수집
데이터베이스	칸마다 라벨이 붙은 물류 창고	Supabase PostgreSQL
RLS	회사별로 잠긴 창고 칸	다른 팀 데이터는 보이지 않음
인증(Auth)	사원증을 찍고 들어가는 출입문	팀원 로그인
JSON 출력	빈칸이 정해진 주문서에 답을 적게 하기	위험 표현 / 사유 / 대체안을 정해진 칸에
멀티모달	글뿐 아니라 사진도 읽는 직원	상품 이미지에서 성분표 읽기
Vector DB / RAG	뜻으로 책을 찾아 펼쳐주는 사서	약기법 사례가 많아진 뒤
배포	가게 문을 열고 간판을 거는 일	Vercel에 올리기
스케줄러	매일 밤 정해진 시간에 오는 납품 트럭	GitHub Actions
백그라운드 작업 큐	주문을 받아두고 컨베이어에 올려 차례로 처리	리뷰 수천 건 분석 (확장 단계)

이 분석이 따른 원칙

도구를 먼저 고르지 않고, 문제를 정의하고 → 필요한 기능을 찾고 → 그 기능에 맞는 언어와 도구를 골랐습니다. 이 문서의 선택을 그 흐름으로 다시 읽으면 이렇습니다.

- 팀이 표와 차트를 함께 본다 → 웹 → TypeScript + Next.js
- 쇼핑몰 리뷰를 모아야 하고 일본어를 자른다 → 크롤링·형태소 분석 → Python
- 경쟁사 비교와 원문 근거가 표끼리 연결된다 → 관계형 DB → PostgreSQL
- 규제 문서가 아직 양이 적다 → Vector DB는 나중에
- 서버 관리보다 기능에 시간을 써야 한다 → 관리형 → Supabase + Vercel
- 긴 규제 문서와 일본어 뉘앙스를 판단하고 상품 이미지도 읽어야 한다 → 긴 문서 이해·이미지 입력·구조화 출력이 되는 LLM → Claude API (Gemini·GPT도 가능)

요구사항이 바뀌면 도구도 같은 흐름으로 다시 고르면 됩니다. 예를 들어 팀에 Python만 다루는 사람뿐이라면, 화면을 Streamlit으로 바꾸고 언어를 하나로 줄이는 것이 더 나은 선택이 될 수 있습니다.